UNIVERSITY OF BOTSWANA

CSI315 – WEB TECHNOLOGY AND APPLICATIONS

Cookies and Sessions

Introduction

- In this lecture we are going to look at the ways PHP can resolve the problem of HTTP's statelessness.
  - This is a necessary aspect of creating a dynamic web-based application.
- Persistence of user data is tremendously important.
  - No-one wants to have to login every time they perform an action on an application.
- We will also discuss the ways in which our PHP scripts can deal with incoming information from HTML pages and URL queries.

Statelessness - 1

- The problem with HTTP as a delivery platform is that it is ***stateless***.
  - The only data you have in the form is the data you take with you.
- Traditionally, this problem is solved by using ***cookies*** or ***sessions***.

Statelessness - 2

- HTTP permits the sending of data to web pages.
  - This data is however not sent onto other pages due to the statelessness of the protocol.
- Two methods for this are provided:
  - ***GET***
  - ***POST***
- When it is time to send information (for example, from form elements), it is encoded by the client and then sent in one of these two ways.

GET

- Using the GET method, the information that is encoded gets sent as an extension to the URL.
  - It will appear as something like: http://<url>/dice_roll_get.php**?num=6&faces=7**
- This information is available to PHP via the $_GET variable.
  - The action used to provide data to a PHP form influences the code that we use to access it.
- We can make use of the GET protocol by changing the ***action*** in our form to GET.

Example Using GET - HTML

```
<html>
  <head>
    <title>Dice Form</title>
  </head>
  <body>
  <form action = "dice_roll_get.php" method = "get">
     <p>How many dice</p>
     <input type = "text" name = "num">
     <p>How many faces?</p>
     <input type = "text" name = "faces">
     <input type = "submit" value = "Roll">
     <input type = "reset" value = "Clear values">
  </body>
</html>
```

Example Using GET - PHP

```php
<?php
    $num = $_GET["num"];
    $faces = $_GET["faces"];
    $total = 0;
    $roll = 0;

    for ($i = 0; $i < $num; $i++) {
      $roll = (rand()%$faces) + 1;
      echo "<p>Dice roll " . ($i+1) . " is $roll.</p>";
      $total += $roll;
    }
    echo "<p>Total roll is $total</p>"
?>
```

Overview of GET

- There are restrictions on how much information can be sent using GET.
  - And on the type of information.
- It can send a maximum of 1024 characters.
- It cannot send binary data, only alphanumeric characters.
- It should never be used to send sensitive data, such as passwords.
  - They get encoded into the URL.

Why Use GET?

- GET is a somewhat limited protocol, but it has one very important benefit.
  - It lets you send data to a server with a URL only.
  - This is very important if you want to make access to a web API as simple as possible.
- There is no need for a front end HTML page to the PHP program we just saw.
  - You can manipulate it through URLs entirely.
- This is something the Post protocol does not do as easily.

The POST Protocol

- The POST protocol is most useful on a day-to-day basis.
- POST has no limitations on size of data.
- It has no limitations on data types.
  - You can use it to send binary data too.
- It works by placing the encoded data in a standard HTTP header.
  - As such, the data does not appear in the URL.

The Limitations of POST and GET

- Both of these protocols permit you to send data to a PHP script.
- That data persists only as long as the script is running.
  - If we reload a page that contains a script, it will usually ask if we want to resend the data.
- If we move outside the confines of a single PHP script, we will lose the data.
- That is a consequence of HTTP's statelessness.

Cookies and Sessions - 1

- Cookies are little files stored on a user's computer that contain certain pieces of information.
  - They are then read in by a web page and accessed to ensure data can be available between pages.
- Sessions fulfill the same role, but most of the information does not get stored on a user's computer.
  - It is available only as long as their browser is open and the session is active.

Cookies

- When using cookies, we must declare them ***before*** any of the HTML in a script.
  - This is because they are part of an HTTP header rather than part of the content.
- Cookies are available on the ***next page load***.
  - You cannot set and access a cookie in the same pass.
- Cookies are set using the `setcookie` function.
  - This takes two parameters – a name for the cookie and its value.
    - You can add a third to define an expiration time.

Cookie Example

```
<?php
        $thetext = $_POST["mytext"];
        setcookie ("texttokeep", $thetext,  time() + 10000);
?>
<html>
  <head>
    <title>Cookie Page</title>
  </head>
  <body>
  <?
        echo "<p>The post text   " . $_POST["mytext"] .
           ", we won't be able to pass that on.</p>";
  ?>

  <a href = "cookie2.php">Onto the next page</a>

  </body>
</html>
```

The Next Page – cookie2.php

```
<html>
  <head>
    <title>Passed it on</title>
  </head>
  <body>

  <?php
        echo "<p>The post text is " . $_POST["mytext"] .
            ", we didn't get that passed on.</p>";
        echo "<p>The text is still " . $_COOKIE["texttokeep"] .
            ", as we know from cookies.</p>";
  ?>

  </body>
</html>
```

Manipulating Cookies

- We can change the value of a cookie by altering it directly in the `$_COOKIE` variable:
  - `$_COOKIE["texttokeep"] = "Hello World";`
- We can delete a cookie by setting its expiry date to be in the past:
  - `setcookie ("texttokeep", "", time() - (60 * 60));`
- We can check to see if a cookie was accepted by checking the return value:
  - `If (setcookie ("texttokeep", "blah") == TRUE) {`

Limitations of Cookies

- There are limitations to cookies.
  - Not all clients support them.
  - Not all users will accept them.
- They are meant for infrequent sending of small pieces of information.
  - The real work of your application should happen on the server.
- They can only hold a small amount of information each.

Sessions - 1

- Sessions fill the same basic role as cookies.
  - Getting around the statelessness that is inherent in HTTP.
- Sessions are managed by a pair of cookies.
  - One on the server
  - One on the client
- The client cookie contains only a reference to a session stored on the server.
  - The server thus manages the data for that session.

Working with Sessions

- To setup a session, we use the `session_start` function of PHP.
  - As with a cookie, this must come ***before*** any HTML is sent to the browser.

```
<?php
  session_start();
?>
```

Sessions - 2

- Once you have a session open, you can register something as being a session variable, like so:
  - `$_SESSION[“mytext”] = $mytext;`
- This makes sure that the `mytext` variable is available to any other pages making use of the session.
- The variables are stored in the `$_SESSION` variable in the same way that cookies are.

Sessions Example

```php
<?php
    session_start();
?>
<html>
  <head>
    <title>Session Page</title>
  </head>
  <body>
  <?php
      $mytext = $_POST["mytext"];

      echo "<p>The post text is $mytext and we'll register that
              in a session.</p>";
      $_SESSION["mytext"] = $mytext;
  ?>
  <a href = "session_next_page.php">Onto the next page</a>

  </body>
</html>
```

Session_next_page.php

```php
<?php
        session_start();
?>

<html>
  <head>
    <title>Passed it on</title>
  </head>
  <body>
  <?php
        echo "<p>The session variable mytext is  " .
           $_SESSION["mytext"] . ".</p>";
  ?>

  </body>
</html>
```

Manipulation of Sessions

- Once a session has been created, it is relatively simple to manipulate.
  - Most of it is done through the `$_SESSION` variable.
- If you wish to delete session data, you can use the unset function:
  - `unset ($_SESSION["something_sensitive"]);`
- You can destroy a session completely using `session_destroy.`

Cookies and Sessions - 2

- In the end, which you choose is based on several factors:
  - Does the client accept cookies?
    - If not, you will need sessions.
  - Do you want to store user data over a significant period of time?
    - If you do, you will need cookies.
- For the system we develop through this course, you will use both.

Conclusion

- HTTP is a stateless protocol.
  - Which makes it a little difficult to make dynamic web pages.
- PHP offers cookies and sessions as a way to resolve this problem.
  - There are two ways of accomplishing the same basic goal.

Terminology

- ***Cookie***
  - A small piece of data stored on a user's computer to ease dynamic application development.

- ***Session***
  - A temporary mapping between the state of a server and a client's system.